

Implementation Plan and Commit Blueprint

Offline Field Issue Tracker

1. Six-Hour Schedule

The schedule uses eight 45-minute blocks. Each block ends with a demonstrable checkpoint; the final 30 minutes are reserved for integration, polish, and packaging.

Time	Focus	Deliverable / checkpoint
00:00–00:45	Bootstrap and contracts	App shell, API types, enums, lint/test scripts
00:45–01:30	Data model and local store	Dexie tables, UUID creation, transactional draft save
01:30–02:15	Field form	Validation, GPS/manual location fallback, draft/submit UI
02:15–03:00	Queue and connectivity	Network banner, queue manager, retry classifier/backoff
03:00–03:45	Backend persistence/API	SQLite migration, sync endpoint, idempotency constraint
03:45–04:30	FSM and coordinator UI	Transition service, filters, detail/history, rejection modal
04:30–05:15	Integration and resilience	Browser/API integration tests, offline/reconnect scenarios
05:15–06:00	QA, accessibility, docs	Manual QA, keyboard/contrast pass, final docs and demo script

Block exit criteria

- **Block 1:** A clean install starts the app and test runner.
- **Block 2:** A saved draft survives reload.
- **Block 3:** A worker can submit a complete report offline.
- **Block 4:** Queue states and manual Retry are visible.
- **Block 5:** Repeated idempotent sync creates one server report.
- **Block 6:** Invalid transitions and rejection reasons are enforced.
- **Block 7:** Automated tests cover the critical offline path.
- **Block 8:** A fresh reviewer can execute the assessment demo without undocumented setup.

2. Atomic Git Commit Strategy

Use small commits that each build and have one reviewable purpose.

#	Conventional commit	Scope
1	chore: bootstrap offline issue tracker workspace	Tooling, scripts, environment configuration
2	feat(schema): add report lifecycle and persistence types	Shared enums, DTOs, validation schemas
3	feat(storage): add Dexie offline report and queue tables	IndexedDB schema and repository
4	feat(field): add offline report capture form	Form fields, GPS/manual fallback, local save
5	feat(sync): add connectivity observer and retry worker	Queue processing, backoff, status indicators
6	feat(api): add SQLite report persistence and sync endpoint	REST route, transaction, idempotency
7	feat(workflow): enforce report state transitions and audit history	FSM, transition endpoint, history writes
8	feat(coordinator): add filtering and review dashboard	Desktop queue, detail panel, rejection modal
9	test(sync): cover idempotent retries and partial batch outcomes	API integration and client sync tests
10	test(workflow): cover valid and invalid lifecycle transitions	FSM unit tests and authorization cases
11	fix(ui): preserve failed queue items and correction context	Error recovery and regression fix
12	docs: add offline tracker technical assessment documentation	Six publication-ready documents and demo notes

Do not squash these commits before assessment unless the rubric requires a single commit; their sequence demonstrates incremental engineering judgment.

3. Manual QA Checklist

Setup

- ☐ Start the frontend and backend using the documented commands.
- ☐ Open the application in a Chromium browser.
- ☐ Open DevTools → Application and confirm IndexedDB is available.
- ☐ Open DevTools → Network and enable “Offline” under throttling.

Offline capture and durability

- ☐ With Network Throttle → Offline, open the field form.

- ☐ Create a report with GPS unavailable and enter a manual landmark.
- ☐ Confirm required-field errors prevent submission without clearing values.
- ☐ Save a draft and verify the **Draft** badge.
- ☐ Reload the page while still offline; verify the draft remains.
- ☐ Submit the report; verify **Pending Sync**, not **Synced**.
- ☐ Close and reopen the tab; verify the queue item remains.
- ☐ Double-click Submit or use rapid taps; verify one local report and one queue command.

Sync recovery

- ☐ Switch DevTools back to Online.
- ☐ Confirm the banner changes to **Online**, then **Syncing**.
- ☐ Verify the report becomes **Synced** and displays server time.
- ☐ Refresh the page and verify it is not resubmitted.
- ☐ Turn Offline during an in-flight request; verify the item remains queued.
- ☐ Restore Online and use the queue's manual **Retry**; verify recovery.

Idempotent retries

- ☐ Capture the outgoing **POST /api/reports/sync** request.
- ☐ Replay it with the same **Idempotency-Key**.
- ☐ Verify a successful replay response and exactly one **reports** row.
- ☐ Replay with the same key but materially different payload; verify 409.
- ☐ Send a batch containing one valid and one invalid item; verify per-item outcomes.

Error handling

- ☐ Simulate a 422 response; verify **Failed**, field-level correction guidance, and no automatic retry loop.
- ☐ Simulate a 409 stale transition; verify the detail view refreshes and explains the conflict.
- ☐ Simulate a 500; verify retry/backoff and visible **Sync Error**.
- ☐ Verify malformed server responses do not mark a report as synced.

Coordinator workflow

- ☐ Filter reports by status, priority, and category.
- ☐ Move **Submitted** → **Assigned** with an assignee.
- ☐ Move **Assigned** → **In Progress** → **Resolved**.
- ☐ Reject from **Submitted** and **Assigned** with a reason.
- ☐ Attempt **Draft** → **Resolved**, **Rejected** → **Assigned**, and **Resolved** → **In Progress**; verify rejection.
- ☐ Attempt rejection without a reason; verify 422 and no history mutation.
- ☐ Open history and verify chronological, append-only entries.

Accessibility and ergonomics

- ☐ Navigate the core flow using keyboard only.
- ☐ Confirm focus is visible and modal focus is trapped.
- ☐ Use a contrast checker against primary text, status, and action controls.
- ☐ Confirm every touch target is at least 48×48 CSS px.

- ☐ Zoom to 200% and verify no essential action is inaccessible.
- ☐ Confirm status is conveyed by text/icon/pattern, not color alone.

4. Assessment Demo Script

1. Start online and show the coordinator list.
2. Switch to Offline and create a report with manual location.
3. Reload to prove local durability.
4. Show the queue's **Pending Sync** state.
5. Restore connectivity and show automatic synchronization.
6. Replay the sync request and show idempotent behavior.
7. Open the coordinator dashboard, assign the issue, reject a second issue with a reason, and inspect history.
8. End on the API contract and test output, emphasizing zero duplicate records and explicit failure handling.